第一章

1. 文件系统和数据库系统的区别？两者适合的用途？
    1. 文件系统的数据面向某一应用，**共享性差、冗余度大、独立性差**，文件的记录虽然是有结构的，但是整体无结构。
    2. 数据库系统的数据不再仅仅面向某一个应用，而是面向整个组织或企业。数据的共享性高，冗余度小，独立性较高，数据库中的数据用数据模型**组织、描述和储存**，由数据库管理系统提供安全性、完整性、并发控制和恢复能力。
    3. 功能比较简单、固定的应用系统适合用**文件系统**。如照片、短信、微信数据等。几乎所有企业或部门的**信息系统**都以**数据库系统**为基础。
2. 数据管理有几种方式？3 各自的特点是什么？
    1. 人工管理面向某一应用，无共享、不独立、无结构。
    2. 文件系统的数据面向某一应用，共享性差、冗余度大、独立性差，文件的记录虽然是有结构的，但是整体无结构。
    3. 数据库系统的数据不再仅仅面向某一个应用，而是面向整个组织或企业。数据的共享性高，冗余度小，独立性较高，数据库中的数据用数据模型**组织、描述和储存**，由数据库管理系统提供**安全性、完整性、并发控制和恢复能力。**
3. 为什么需要数据库？
    1. 相比人工管理和文件系统，数据库系统的数据不再仅仅面向某一个应用，而是面向整个组织或企业。数据的共享性高，冗余度小，独立性较高，数据库中的数据用数据模型**组织、描述和储存**，由数据库管理系统提供**安全性、完整性、并发控制和恢复能力。**
4. 数据库的主要功能？数据库用来做什么？数据库有什么特点？（4）
    1. 数据结构化。**整体结构化**（与文件系统的本质区别）
    2. 共享度高，冗余度小，易扩充
    3. 数据独立性高：物理存储+逻辑结构
    4. 由DBMS统一管理和控制，提供**安全性、完整性、并发控制和恢复能力。**
5. 数据库管理系统是什么？位于**用户与操作系统之间**的一层**数据管理软件。**
6. > 数据库管理系统的功能？（选择题，不用背）
    >
    > 1. 数据定义功能（DDL）
    > 2. 数据组织、存储和管理
    > 3. 数据操纵功能（DML）
    > 4. 数据库的事务管理和运行管理
    > 5. 数据库的建立和维护功能
    > 6. 其它功能，如不同数据库之间的互访和互操作功能。
7. 数据库系统的组成是什么？
    1. 数据库：软硬件性能越高越好
    2. 数据库管理系统（及其应用开发工具，如DMLDDL编译器）
    3. 应用程序
    4. 数据库管理员：管理两类共享资源——数据库和数据库管理系统软件
8. 数据模型的定义是什么？有什么组成要素？
    1. 数据模型是对现实世界数据特征的抽象，用以**抽象、表示和处理**现实世界中的数据和信息
    2. 有3个组成要素：
        - 数据结构（刻画模型性质的最基本方面，静态特性）
        - 数据操作（查询+更新，动态特性）
        - 数据的完整性约束条件

3. 是数据库系统的核心和基础，通常按照数据结构的类型来命名

9. 概念/逻辑/物理模型分别完成的任务？功能？

    1. 概念模型：按**用户**的观点来对数据和信息建模，用于数据库设计（E-R图表示）
    2. 逻辑模型：按**计算机系统**的观点对数据建模，用于DBMS实现
    3. 物理模型：对数据**最底层的抽象**，描述数据在**系统内部**的表示方式和存取方法。

10. 概念模型的定义和组成？

    - 按**用户的观点**来对数据和信息建模，用于数据库设计（E-R图表示）
    - 实体
    - 属性：注意**实体和联系**都可以有属性！
    - 码：唯一标识实体
    - 实体型：**实体名+属性名**，如学生（学号，姓名，性别）是一个实体型
    - 实体集：同一类型**实体的集合**

11. **关系模型**（Relational Model)）重点

    - 关系：一个表
    - 元组：一行
    - 属性：一列
    - 码：某个属性组，可以**唯一确定**一个元组
    - **域：属性的取值范围**
    - **分量：元组中的一个属性值**
    - 关系模式：对关系的描述，一般表示为关系名（属性1，属性2...）

12. 关系模型的优缺点？

    1. 优点：

        1. 具有严格的数学基础。
        2. 关系模型的概念单一。
        3. 关系模型的存取路径对用户**透明**，从而具有更高的数据独立性、更好的安全保密性，也简化了程序员的工作和数据库开发建立的工作。

    2. 缺点：由于存取路径对用户透明，查询效率往往远不如非关系数据模型。

13. 试述数据库系统三级模式结构，并说明这种结构的**优点**是什么？

    - 模式：数据库中全体数据的**逻辑结构和特征**的描述，是所有用户的公共数据视图。
    - 外模式：数据库用户的数据视图，是与**某一应用有关**的数据的逻辑表示。
    - 内模式：数据**物理结构和存储方式**的描述，是数据在数据库内部的组织方式。

    优点：由**外模式/模式映像**和**模式/内模式映像**这两层映像保证数据库系统中的数据能够具有较高的**逻辑独立性**和**物理独立性**。

14. 二级映像是什么？

    1. 外模式/模式映像

        - 保证数据的**逻辑独立性**：模式改变时外模式可以不变，而应用程序依据外模式编写，所以保证了数据与程序的逻辑独立性

    2. 模式/内模式映像

        - 保证数据的**物理独立性**：底层存储结构变，模式可不变，应用程序随之也不变

第二章

1. 关系操作包括：插入、删除、查询和修改。

第三章 sql语法

1. sql和java区别：sql面向集合、java面向对象
2. > sql的特点：
   >
   > 1. 综合统一
   > 2. 高度非过程化
   > 3. 面向**集合**的操作方式
   > 4. 以同一种语法结构提供多种使用方式
   > 5. 语言简洁，易学易用
   >   - 数据定义：CREATE，DROP，ALTER
   >   - 数据查询：SELECT
   >   - 数据操作：INSERT，UPDATE，DELETE
   >   - 数据控制：GRANT，REVOKE
3. 层次化数据库的结构：
   1. 一个DBMS的实例可以建多个数据库
   2. 一个数据库可以建立多个模式
   3. 一个模式包括多个表、视图和索引等数据库对象
4. RESTRICT：删除有限制，被依赖就不能删。CASCADE：自身和依赖对象都被删除。
5. 视图有哪些作用？使用视图的理由？
   - 简化用户的操作
   - 使用户能以多种角度看待同一数据
   - 视图对重构数据库提供了一定程度的**逻辑独立性**
   - 对机密数据提供安全保护
   - 可以更清晰的表达查询

   > 注意视图是虚拟表，描述关系，不存储数据，不都能进行增删改查
6. 哪些事情视图做不到？
   - **行列子集视图**是可更新的，但部分特殊视图不允许更新。
   - 视图消解法有局限性，有时不能正确地查询。

第四章

1. DAC（C1）：定义各个用户对不同数据对象的存取权限。
2. MAC（B1）：每个主体被标以**许可证级别**，每个客体被标以**密级。**通过**对比主客体标记**，确定主体是否能存取客体。
3. 强制存取规则的客体和主体为什么这么设计？
   1. 主体的许可证级别>=客体的密级----->可读取
   2. 主体的许可证级别<=客体的密级----->可写：用户可以对写入的对象赋予**高于自己的密级**，否则密级可以高流向低，造成数据泄露
   3. 防止高级别泄露，防止低级别查看
   4. 可修改：读+写，所以可修改的数据只有密级相同的

第五章 完整性

1. 触发器：
   1. 和一般sql的区别：是用户定义在关系表上的一类**由事件驱动**的特殊过程
   2. 放在本地还是远端？**远端**，慢但可重用
   3. 分为行级、语句级触发器

第六章 范式（*）

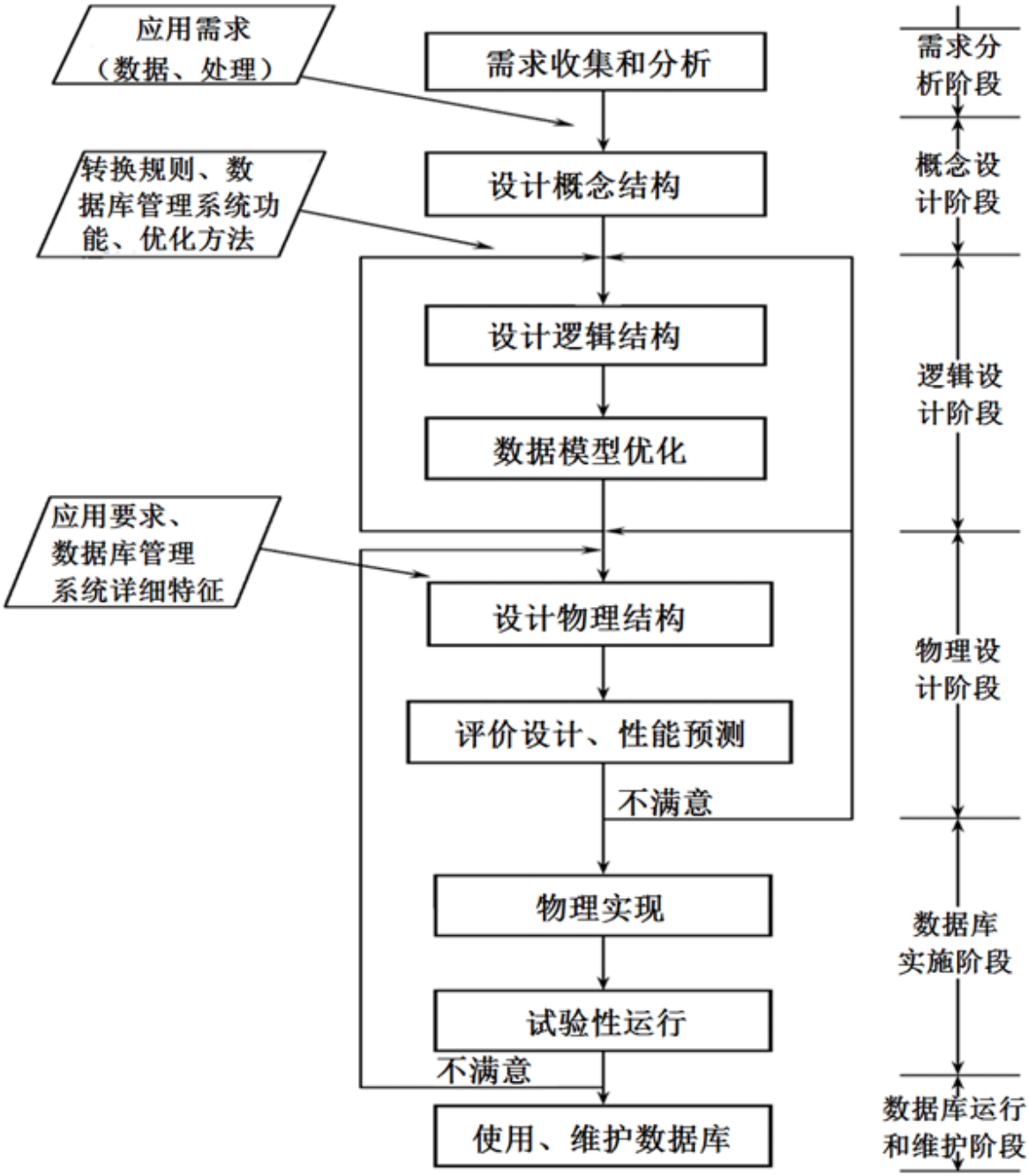
1. 1NF：R的所有属性都是**不可分**的基本数据项
2. 2NF：R∈1NF，并且每一个**非主属性**都**完全函数依赖**于任何一个候选码
3. 3NF：每个非主属性既不**传递依赖**于码，也不**部分依赖**于码。（不能有传递依赖）
4. BCNF：若X->Y是非平凡依赖，X必含有码。即每个**决定属性集**（出现在->左边）都包含**候选码**。

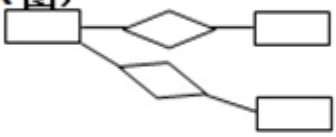
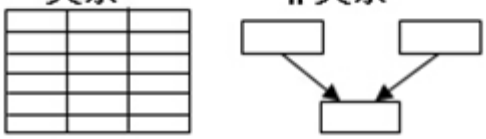
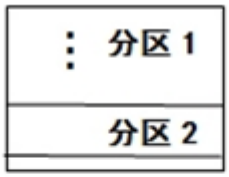
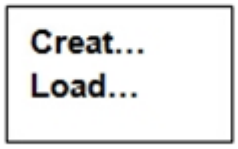
1NF	2NF	3NF	BCNF
数据冗余	变少了		
插入异常	有改进		消除
删除异常	仍是大问题	有改进	消除
更新异常	有改进		

> 如果没有非主属性，显然满足3NF。

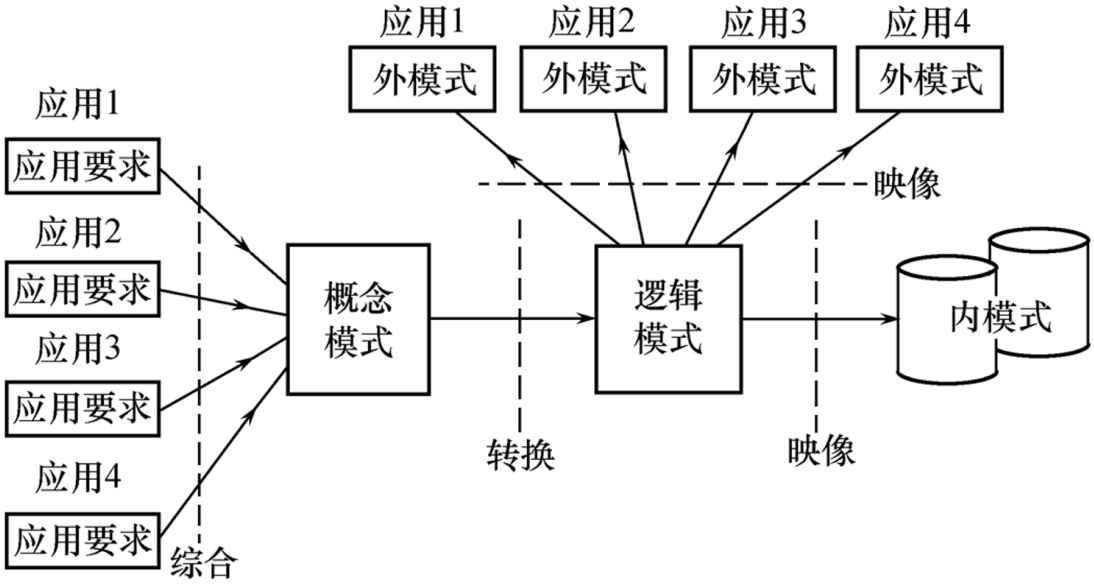
第七章 数据库设计

1. 数据库设计步骤有哪些?
    1. 需求分析
    2. **概念结构设计**：形成概念模型
    3. 逻辑结构设计：转换为数据模型
    4. 物理结构设计：选择物理结构
    5. 数据库实施
    6. 数据库运行和维护


应用需求
（数据、处理）
需求收集和分析
需求分析阶段
转换规则、数据库管理系统功能、优化方法
设计概念结构
概念设计阶段
设计逻辑结构
数据模型优化
逻辑设计阶段
应用要求、数据库管理系统详细特征
设计物理结构
评价设计、性能预测
物理设计阶段
不满意
物理实现
试验性运行
数据库实施阶段
不满意
使用、维护数据库
数据库运行和维护阶段

设计阶段	设计描述
需求分析	数字字典、全系统中数据项、数据结构、数据流、数据存储的描述
概念结构设计	概念模型（E-R 图）  数据字典
逻辑结构设计	某种数据模型 关系 非关系 
物理结构设计	存储安排 存取方法选择 存取路径建立 
数据库实施	创建数据库模式 装入数据 数据库试运行 
数据库运行和维护	性能监测、转储/恢复、数据库重组和重构

2. 数据库设计过程中的各级模式
    1. 需求分析阶段（最左）：综合各个用户的应用需求
    2. 概念设计阶段：形成独立于机器特点，独立于各个数据库管理系统产品的**概念模式**（E-R图）
    3. 逻辑设计阶段：生成DBMS支持的数据模型
        1. 首先将E-R图转换成具体的数据库产品支持的数据模型，如关系模型，形成数据库**逻辑模式**
        2. 然后根据用户处理的要求、安全性的考虑，在基本表的基础上再建立必要的视图（View），形成数据的**外模式**
    4. 物理设计阶段（最右）：根据数据库管理系统特点和处理的需要，进行物理存储安排，建立索引，形成数据库**内模式**

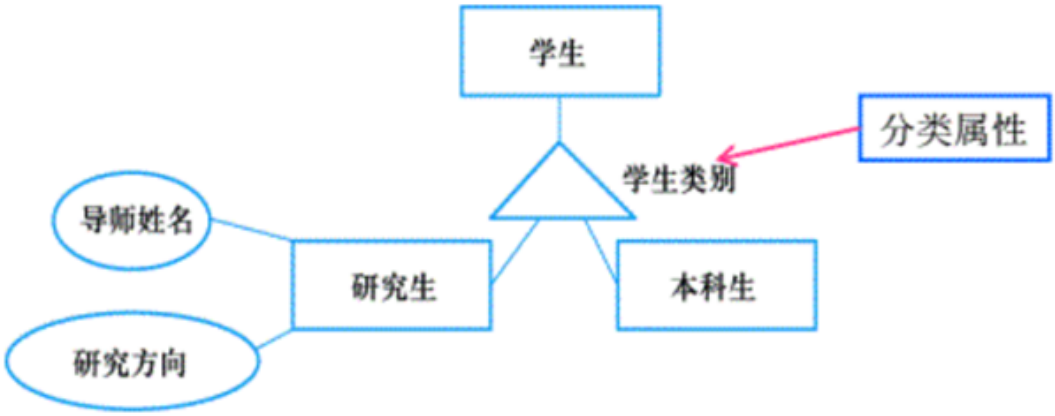


3. 数据字典的内容?

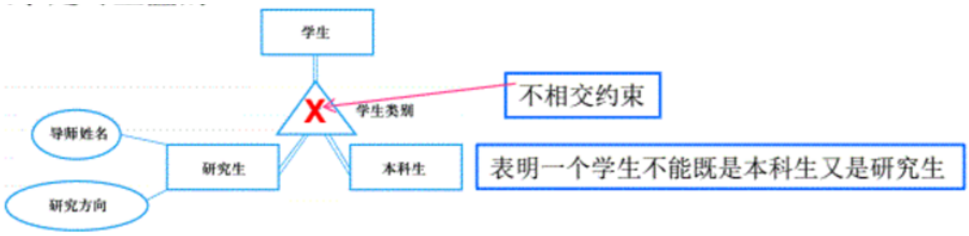
    1. 数据项（数据的最小组成单位）
    2. 数据结构（由若干个数据项组成）
    3. 数据流：**数据结构**在系统内传输的**路径。**
    4. 数据存储：数据结构**停留或保存**的地方，也是数据流的来源和去向之一。
    5. 处理过程：一般用判定表或判定树来描述。

4. ISA联系：

    1. **分类属性**是父实体型的一个属性，其值把父实体型中的实体分派到**子实体型**中



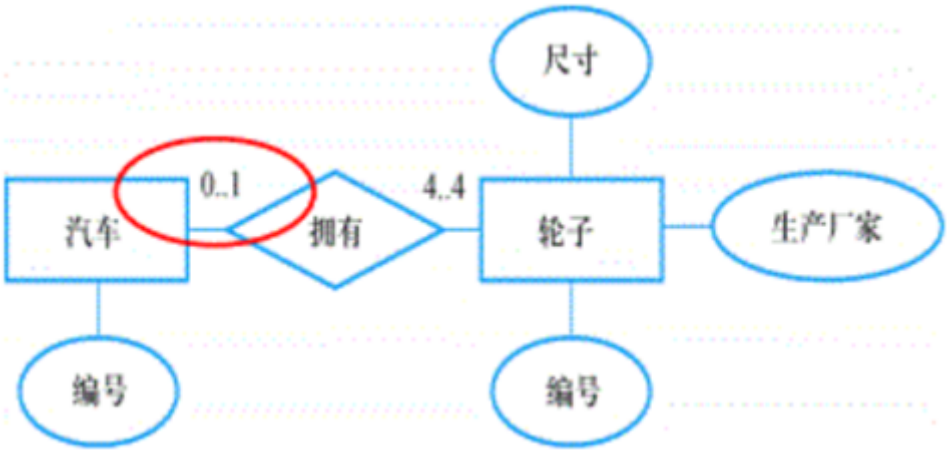
    - 不相交约束：父类中的一个实体**不能同时属于**多个子类中的实体集。



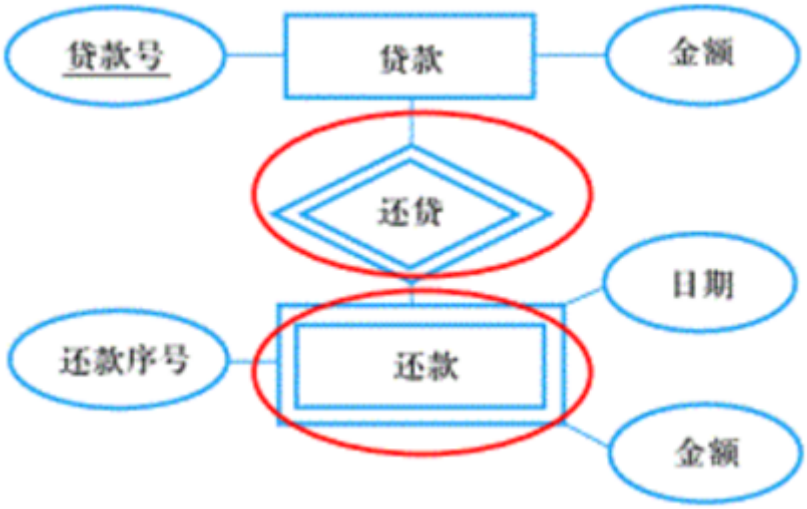
    - 完备性约束：父类中的一个实体是否必须是某一个子类中的实体。若是，则叫做完全特化。用**双线连接**表示。



5. 基数约束：最少次数和最多次数 a..b
    1. 强制参与约束：min=1，如1..1
    2. 非强制参与约束：min=0，如0..*
6. Part-of联系：
    - 非独占（聚合）：整体被破坏，另一部分实体仍可以存在；用**0..1**即可表示，如汽车和轮子

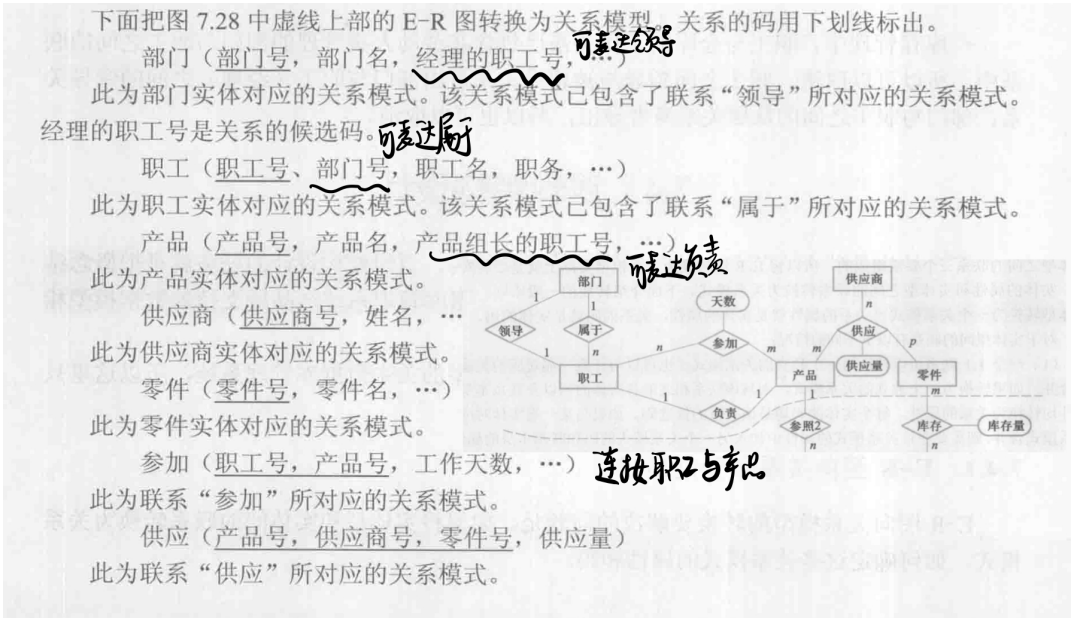


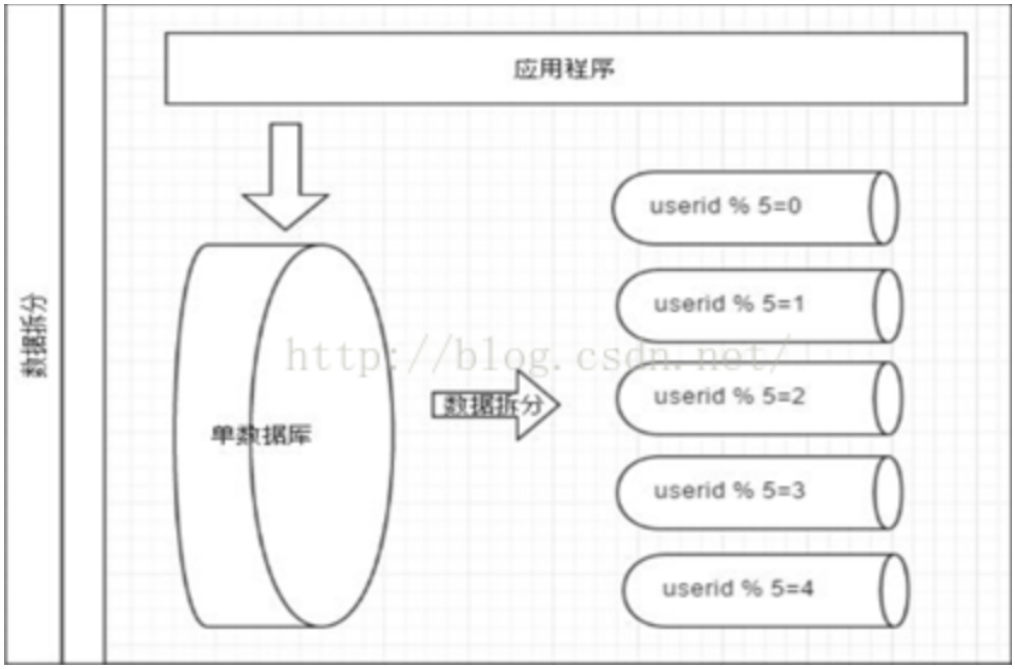
    - 独占（组合）：整体实体如果被破坏，部分实体不能存在；用弱实体类型和识别联系表示
    - **双矩形**表示弱实体型，用**双菱型**表示识别联系，如果贷款没了还款也没了，所以贷款独占还款，还款是弱实体型



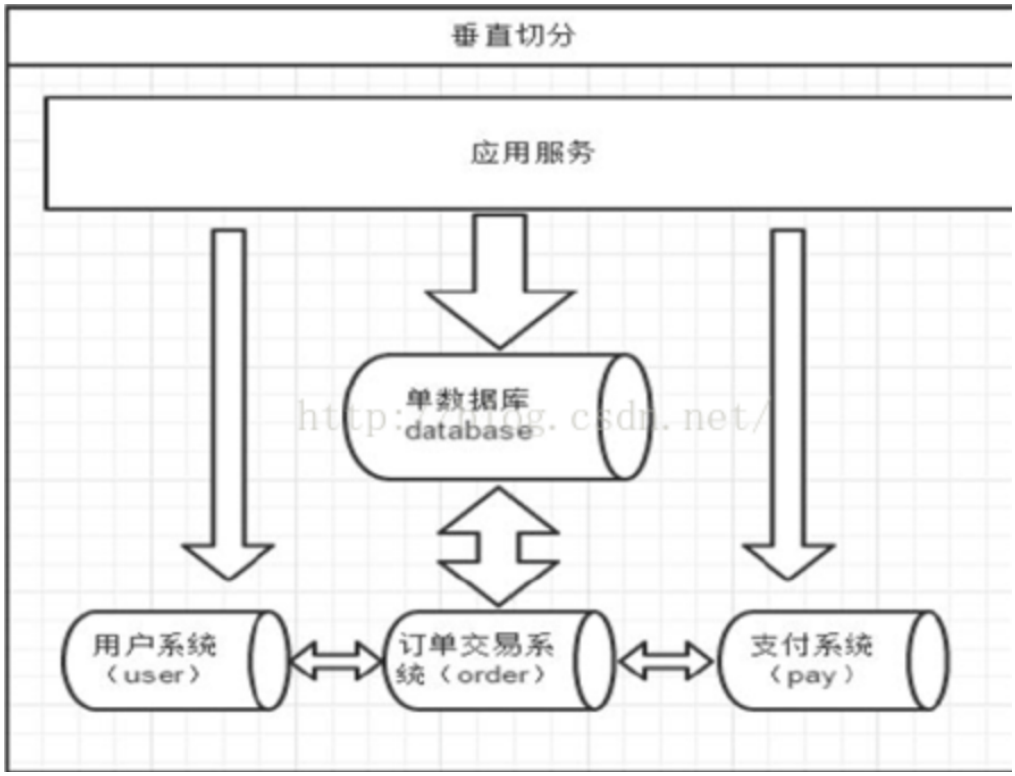
7. ER图向关系的转化掌握

1. 如图中**供应和参加虽然是联系**，但有属性，所以作为实体了。**联系会拥有相连实体的主码。**



8. 水平分解和垂直分解的过程?
    1. 水平分解：把(基本)关系的**元组**分为若干子集合，定义每个子集合为一个子关系。（拿几行出来)
        - 对符合“80/20原则”的，把经常被使用的数据（约20%）水平分解出来，形成一个子关系。
        - 

    2. 垂直分解：R的**属性分解**为若干子集合，形成若干子关系模式。（拿几列出来，比如1NF到2NF)

1. 

9. 存取方法如何选择？（原理）
    - B+树索引存取方法
        - 选择索引存取方法的主要内容：根据应用要求确定对哪些属性列**建立索引**、对哪些属性列建立组合索引、对哪些索引要设计为唯一索引
    - Hash索引存取方法：
        - 如果一个关系的属性主要出现在**等值连接条件**中或主要出现在**等值比较选择**条件中，而且满足下列两个条件之一
            - 该关系的大小可预知，而且不变；
            - 该关系的大小动态改变，但所选用的数据库管理系统提供了动态Hash存取方法。
    - 聚簇存取方法
        - 选择聚簇码，如把同一院系的学生元组集中存放
        - **一个关系只能加入一个聚簇**

            - 为了提高某个属性（或属性组）的查询速度，把这个或这些属性（称为聚簇码）上具有相同值的元组集中存放在连续的物理块中称为聚簇。
            - 该属性（或属性组）称为聚簇码（cluster key）

        -
            - 聚簇的用途：大大提高按聚簇属性进行查询的效率
                - [例] 假设学生关系按所在系建有索引，现在要查询信息系的所有学生名单。
                    - 计算机系的500名学生分布在500个不同的物理块上时，至少要执行500次I/O操作。
                    - 如果将同一系的学生元组集中存放，则每读一个物理块可得到多个满足查询条件的元组，从而显著地减少了访问磁盘的次数。
            - 聚簇既适用于单个关系，也适用于经常进行连接操作的多个关系
                - 把多个连接的元组按连接属性值聚集存放
                - 从而实现多个关系的“预连接”，提高连接操作的效率。

- 选择聚簇存储方法，即确定需要建立多少个聚簇，每个聚簇中包含哪些关系（一个数据库可以建立多个聚簇，一个关系只能加入一个聚簇）
  - 设计候选聚簇
    - 常在一起进行连接操作的关系可以建立组合聚簇
    - 如果一个关系的一组属性经常出现在相等比较条件中，则该单个关系可建立聚簇；
    - 如果一个关系的一个（或一组）属性上的值重复率很高，则此单个关系可建立聚簇。
  - 检查候选聚簇中的关系，取消其中不必要的关系
    - 从聚簇中删除经常进行全表扫描的关系
    - 从聚簇中删除更新操作远多于连接操作的关系
    - 从聚簇中删除重复出现的关系
      - 当一个关系同时加入多个聚簇时，必须从这多个聚簇方案（包括不建立聚簇）中选择一个较优的，即在这个聚簇上运行各种事务的总代价最小。
- 局限性和适合的场景：
  - 聚簇只能提高某些特定应用的性能
  - 建立与维护聚簇的开销相当大
    - 对已有关系建立聚簇，将导致关系中元组的物理存储位置移动，并使此关系上原有的所有索引无效，必须重建。
    - 当一个元组的聚簇码改变时，该元组的存储位置也要做相应改变。
  - 当通过聚簇码进行访问或连接是该关系的主要应用，与聚簇码无关的其他访问很少或者是次要的时，可以使用聚簇
    - 尤其当SQL语句中包含有与聚簇码有关的ORDER BY, GROUP BY, UNION, DISTINCT等子句或短语时，使用聚簇特别有利，可以省去或减化对结果集的排序操作

第十章

1. ACID特性(*)：
   1. 原子性：事务是基本单位，要么都做要么都不做
   2. 一致性：
      1. 若一致，数据库中**只包含**成功事务提交的结果；
      2. 若不一致，数据库系统运行中发生**故障**，有些事务尚未完成就被迫中断；这些未完成事务对数据库所做的修改有一部分已写入**物理数据库**，这时数据库就处于一种不正确的状态
   3. 隔离性：一个事务的执行不被其他并发事务**干扰**
   4. 持续性：一个事务一旦提交，它对数据库中数据的改变就应该是**永久**性的
2. 为什么事务非正常结束时会影响数据库数据的正确性？
   1. 数据库系统运行中如果发生**故障**，有些事务尚未完成就被迫中断；这些未完成事务对数据库所做的修改有一部分已写入**物理数据库**，这时数据库就处于一种不正确的状态
3. 事务故障（仅指以下的非预期故障）
   1. 运算**溢出**
   2. 并发事务发生死锁而被选中**撤销该事务**
   3. 违反了某些**完整性限制**而被终止等

4. 系统故障：软故障，不破坏数据库
    1. 一些尚未完成的事务的结果可能**已送入物理数据库**，造成数据库可能处于不正确状态。
    2. 发生系统故障时，有些已完成的事务可能有一部分甚至全部留在**缓冲区**，尚未写回到磁盘上的物理数据库中，系统故障使得这些事务对数据库的修改部分或全部丢失。
5. 介质故障：硬故障，破坏数据库或部分数据库，影响正在存取这部分数据的**所有事务**
6. 计算机病毒：破坏数据库

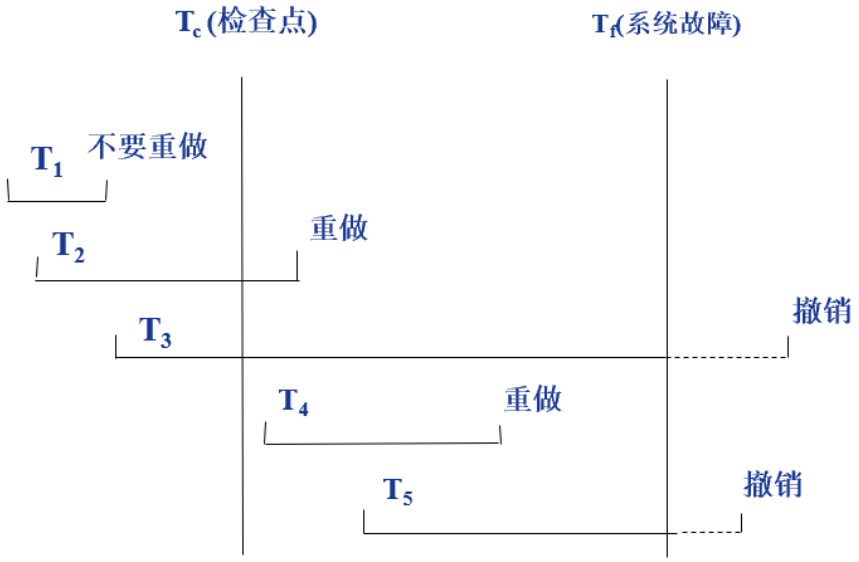
故障类型	**恢复策略**
事务故障	强行回滚该事务
系统故障	1.强行撤销所有**未完成**事务 2.重做所有**已提交**的事务
介质故障、计算机病毒	1.重装后备副本 2.利用**日志文件**恢复事务（重做已完成的事务）

1. 必须先写日志文件，后写数据库?
    1. 如果先写了数据库修改，而在运行记录中没有登记这个修改，则以后就**无法恢复**这个修改了。
    2. 如果先写日志，但没有修改数据库，在恢复时只不过是多执行一次UNDO操作，并不会影响数据库的正确性。
2. **恢复策略**
    1. 事务故障的恢复：恢复到**最开始**，系统自动完成
        1. 反向扫描文件日志
        2. 对该事务的**更新操作**执行**逆操作**。即将日志记录中“更新前的值” 写入数据库。
        3. 如此处理下去，直至读到此事务的**开始标记**，事务故障恢复就完成了。
    2. 系统故障的恢复：恢复到**故障时**
        1. **正向扫描日志文件**（即从头扫描日志文件），建立REDO队列（**已经提交的事务**）和UNDO队列（**尚未完成的事务**）
        2. 对UNDO队列事务进行**撤销**处理：反向扫描日志文件，执行逆操作
        3. 对REDO队列事务进行**重做**处理：正向扫描日志文件，重新执行登记的操作

        > 撤销**未完成**，重做**已提交**
        >
        > UNDO反向：恢复到第一个失败事务之前
        >
        > REDO正向：恢复到最后一个成功事务之后
        >
        > 只看“提交”与否，**有些日志是回滚仍然要UNDO**

    3. 介质故障的恢复：恢复到**故障时**
        1. **装入最新的后备数据库副本**（离故障发生时刻最近的转储副本），使数据库恢复到最近一次转储时的一致性状态
        2. 装入有关的日志文件副本(转储结束时刻的日志文件副本)，**重做已完成的事务。**
3. 有检查点：**改善恢复效率。**若T在检查点之前COMMIT，检查点的存在可以保证T对数据库所做的修改已写入数据库。只用从最近的检查点扫描日志，节约时间。
    1. 恢复策略：就是看**从检查点到故障中间**有没有提交，有提交就重做（防止已完成未写入），没提交就撤销（防止未完成在缓存）

系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略



第十一章 并发控制

1. 并发操作带来的数据不一致性：
    1. 丢失修改（**被后面的写入覆盖**）
        - 如T2的提交结果**破坏了**（覆盖了）T1提交的结果，导致T1的修改被丢失。
    2. 读"脏"数据（**读了别人写的，但别人撤销了**）
        - 事务T1修改某一数据，并将其写回磁盘
        - 事务T2读取同一数据后， T1由于某种原因被**撤销**ROLLBACK
        - 这时T1已修改过的数据恢复原值， T2读到的数据就与数据库中的数据不一致
        - T2读到的数据就为"脏"数据，即**不正确**的数据
    3. 不可重复读（**读了旧的，别人写了新的**）
        - 事务T1读取数据后，事务T2执行更新操作，使T1无法**再现前一次**读取结果。**校验**时就会出错。
            - T2对其做了修改
            - T2删除了其中部分记录
            - T2插入了一些记录
2. 排它锁：若事务T对数据对象A加上**X锁**，则只允许T**读取和修改**A，其它任何事务都不能再对A加任何类型的锁，直到T释放A上的锁（独占标记）
3. 共享锁：若事务T对数据对象A加上S锁，则事务T可以**读A但不能修改A**，其它事务只能再对A加S锁，而不能加X锁，直到T释放A上的S锁（大家都能读但不能改）

T_2 \ T_1	X	S	–
X	N	N	Y
S	N	Y	Y
–	Y	Y	Y

封锁协议

1. 一级封锁协议
    1. 事务T在修改数据R之前必须先对其加X锁，直到**事务结束才释放。**包括正常结束（COMMIT）和非正常结束（ROLLBACK）
2. 二级封锁协议
    1. 一级封锁协议基础上增加事务T在读取数据R之前必须先对其加S锁，**读完后**即可释放S锁。
3. 三级封锁协议
    1. 一级封锁协议基础上加上事务T在读取数据R之前必须先对其加S锁，直到**事务结束**才释放。

	写	读	解决	未解决
一级封锁协议	加X锁	不加锁	丢失修改	不可重复读、读“脏”数据
二级封锁协议	加X锁	加S锁，**读完后**释放	丢失修改，读“脏”数据	不可重复读
三级封锁协议	加X锁	加S锁，**事务结束后**释放	丢失修改、读脏数据、不可重复读	无

封锁协议小结

- 三级协议的主要区别
    - 什么操作需要申请封锁以及何时释放锁（即持锁时间）
- 不同的封锁协议使事务达到的一致性级别不同
    - 封锁协议级别越高，一致性程度越高

	X锁		S锁		一致性保证		
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不读“脏”数据	可重复读
一级封锁协议		√			√		
二级封锁协议		√	√		√	√	
三级封锁协议		√		√	√	√	√

1. 可串行化(Serializable)调度
    - 和**不并发**（串行运行时）得到的结果一样->正确调度
2. 冲突可串行化（比可串行化更严格）
    1. 冲突操作：仅指T1、T2对同一数据的**读写**操作和**写写**操作
    2. 不能交换（Swap）的动作：同一事务的两个操作和不同事务的冲突操作
    3. 定义：一个调度Sc在保证冲突操作的次序不变的情况下，通过交换**两个事务不冲突操作的次序**得到另一个调度Sc'，如果Sc'是**串行**的，称调度Sc是冲突可串行化的调度

两段锁协议

1. 实现并发调度的**可串行性**，从而保证调度的正确性
2. 指所有事务必须分**两个阶段**对数据项加锁和解锁
    - 在对任何数据进行读、写操作之前，事务首先要获得对该数据的**封锁**：扩展阶段
    - 在**释放**一个封锁之后，事务不再申请和获得任何其他封锁：收缩阶段
3. “两段”锁的含义，事务分为两个阶段
    - 第一阶段是获得封锁，也称为扩展阶段
        - 事务可以**申请**获得任何数据项上的任何类型的锁，但是**不能释放**任何锁
    - 第二阶段是释放封锁，也称为收缩阶段
        - 事务可以**释放**任何数据项上的任何类型的锁，但是**不能再申请**任何锁
4. 若并发事务都遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的；若并发事务的一个调度是可串行化的，**不一定**所有事务都符合两段锁协议。
5. 两段锁协议与防止死锁的一次封锁法：
    - 一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此一次封锁法**遵守两段锁协议**（释放后没得再加的东西了）
        - **注意Slock A、Xlock A后，第一个Unlock针对X锁，第二个Unlock针对S锁**
    - 两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此遵守两段锁协议的事务**可能发生死锁**

14. 考虑 T_1 和 T_2 两个事务。

T_1：R(A)；R(B)；$B=A+B$；W(B)；　T_2：R(B)；R(A)；$A=A+B$；W(A)

① 改写 T_1 和 T_2，增加加锁操作和解锁操作，遵循两阶段封锁协议。

② 说明 T_1 和 T_2 的执行是否会引起死锁，给出 T_1 和 T_2 的一个调度说明之。

答：

①

把操作写出来
即可不用写
调度

T_1	T_2
Slock A	Slock B
R(A)	R(B)
Xlock B	Xlock A
R(B)	R(A)
$B=A+B$	$A=A+B$
W(B)	W(A)
Unlock A	Unlock B
Unlock B	Unlock A

② 可能产生死锁，如下面的调度所示：

要写一种调度

T_1	T_2
Slock A	
R(A)	
	Slock B
	R(B)
Xlock B	
	Xlock A

意向锁

1. 封锁的粒度越小，**并发度较高**，但系统**开销也就越大**
2. **意向锁可以加多个**，粒度逐渐减小



T_1 \ T_2	S	X	IS	IX	SIX	–
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
–	Y	Y	Y	Y	Y	Y

Y=Yes，表示相容的请求　　N=No，表示不相容的请求

(a) 数据锁的相容矩阵

- IS：IX、SIX可以，因为后面会判断（2I）
- IX：IS、IX可以，因为后面会判断（2I）
- SIX：IS可以，因为后面会判断（2I），IX、SIX不行，因为S与IX冲突

第十二章 NoSQL

1. 关系模型的优缺点?
    1. 优点:
        1. 具有严格的数学基础。
        2. 关系模型的概念单一。
        3. 关系模型的存取路径对用户**透明**，从而具有更高的数据独立性、更好的安全保密性，也简化了程序员的工作和数据库开发建立的工作。
    2. 缺点：由于存取路径对用户透明，查询效率往往远不如非关系数据模型。
2. NoSQL的由来
    - 内存中的**数据结构和关系型数据**不匹配，每次处理的过程很麻烦（阻抗失谐）
    - 集成数据库和应用程序数据库的问题（NoSQL是应用程序数据库）
    - 集群问题（让用户自行决定可用性和一致性的平衡）

SQL与NoSQL最大的区别

无模式

- 关系型数据库中，首先必须定义“模式”，然后才能存放数据。
- NoSQL数据库，无模式：
  - “键值数据库"可以把任何数据存放在一个“键”的名下。
  - “文档数据库” 对所存储的文档结构没有限制
  - 在列族数据库中，任意列里面都可以随意存放数据
  - 图数据库中可以新增边，也可以随意向节点和边中添加属性。

3. **聚合无知**：关系型数据库的数据模型中，没有“聚合”这一概念。“图数据库"也是聚合无知的。
4. CAP定理是需要放宽一致性约束的原因。同时满足两个即可。
    1. 一致性
    2. 可用性：如果客户可以同集群中的某个节点通信，那么该节点就必然能够处理读取及写入操作。

       可用性定义为“系统中某个**无故障节点**所接收的每一条请求， 无论成功或失败，都必将**得到响应**。”
    3. 分区耐受性：如果发生通信故障，导致整个集群被分割成多个无法互相通信的分区时(这种情况也叫“ 脑裂”)，集群仍然可用

    > 分布式系统中：只能再一致性和可用性之间妥协，分区耐受性是必须的。

5. 仲裁：保证强一致性R+W>N 写入W>N/2 （W=2，N=2，R=3最常见的）
6.

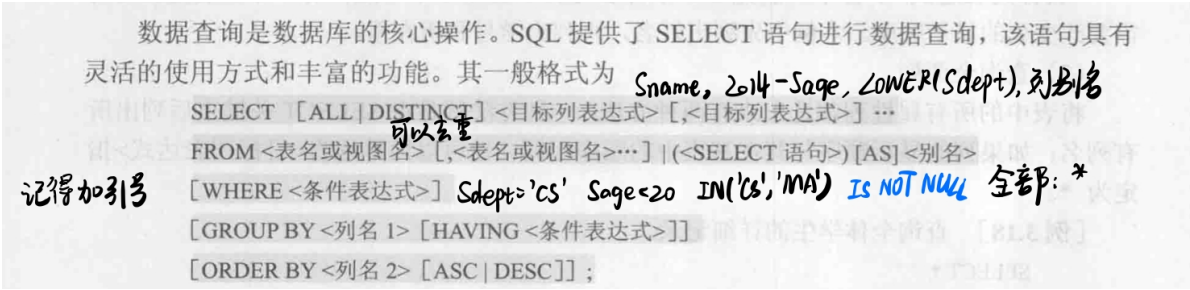
类型	如何完成存放	存的是什么	适合	不适合
键值	写入key、value，若key已有，则更新，否则插入	哈希表，内有key-value	保存会话、购物车、用户配置等量大、不是很重要、不会经常查找的数据	数据间关系（外如码）;含有多项操作的事务;查询数据;操作关键字集合
文档	写入文档	树形结构的"文档"	事件记录(尤其是当事件的数据类型在变);博客平台;网站分析与实时分析;电子商务应用程序	包含多项操作的复杂事务;查询持续变化的聚合结构
列族	写入key，value，数据库会把行键和多列的数据关联起来，并有时间戳	关键字及映射值，并可把值分成多个列族，每个列族代表一张数据映射表	事件记录;博客平台;计数器;限期	需要ACID事务的场合;开发原型早期或者试探某个技术方案;根据查询结果聚合数据（如sum.avg）
图	写入结点、关系（起点、重点、类型、属性）	实体及实体间关系，作为一张有向图	互联数据;安排运输路线、分派货物和基于位置的服务;推荐引擎	更新全部或某子集内的实体（即频繁更新）;图数据库可能没有办法处理数据量过大的场合，即使模型适合。

- 键值:
  - **适用案例**:
    1. 存放会话信息
    2. 用户配置信息
    3. 购物车数据
  - **不适合场合**:
    1. 数据间关系
    2. 含有多项操作的事务
    3. 查询数据
    4. 操作关键字集合
- 文档:
  - 适用案例:
    1. 事件记录
    2. 内容管理系统及博客平台
    3. 网站分析与实时分析
    4. 电子商务应用程序
  - 不适合场合:
    1. 包含多项操作的复杂事务
    2. 查询持续变化的聚合结构
- 列族:

    - 适用案例：
        1. 事件记录
        2. 内容管理系统与博客平台
        3. 计数器
        4. 限期使用
    - 不适合场合：
        1. 需要以“ACID事务”执行写入及读取操作的系统。
        2. 在开发早期原型或刚开始试探某个技术方案时。
- 图：
    - 适用案例：
        1. 互联数据
        2. 安排运输路线、分派货物和基于位置的服务
        3. 推荐引擎
    - 不适合场合：
        1. 图数据库在更新全部或某子集内的实体时也许不适用。
        2. 即便数据模型适合问题领域，某些图数据库可能也无法处理那么大的数据量，尤其在执行“全局图操作”时更是如此。

SQL语句书写

13 单表查询



- 可以是Sno(列)/*(与原表顺序相同)/2014-Sage(表达式)/别名(改变列标题)
- DISTINCT：去掉重复行
- WHERE表示指定条件（作用于表）
    - BETWEEN AND(范围) IN(集合)LIKE(匹配字符)IS NULL(空值)AND OR(多重条件查询)
    - ORDER BY(升降排列)
- 聚集函数 COUNT SUM AVG MAX MIN(需要指定DISTINCT或ALL)

- [例3.41] 查询学生总人数。

  SELECT COUNT(*)

  FROM Student;

- [例3.42] 查询选修了课程的学生人数。（会清除Sno is NULL的）

  SELECT COUNT(DISTINCT Sno)

  FROM SC;

- [例3.43] 计算1号课程的学生平均成绩。

  SELECT AVG(Grade)

  FROM SC

  WHERE Cno= ' 1 ';

- [例3.44] 查询选修1号课程的学生最高分数。

  SELECT MAX(Grade)

  FROM SC

  WHERE Cno='1';

- [例3.45] 查询学生201215012选修课程的总学分数。

  SELECT SUM(Ccredit)

  FROM SC,Course

  WHERE Sno='201215012' AND SC.Cno=Course.Cno;

  - WHERE不能用聚集函数作为条件表达式，聚集函数只用于SELECT和GROUP BY中的HAVING子句。

- GROUP BY按照某一列或多列的值分组（作用于组）
  - 如所有相同Cno值的元组为一组，然后**对每一组作用聚集函数**进行计算。若分组后再进行筛选，则用HAVING指定条件。
  - - [例3.46] 求各个课程号及相应的选课人数。

      SELECT Cno，COUNT(Sno)

      FROM SC

      GROUP BY Cno;

    - 查询结果可能为：

Cno	COUNT(Sno)
1	22
2	34
3	44
4	33
5	48

    - [例3.47] 查询选修了3门以上课程的学生学号。

      SELECT Sno

      FROM SC

      GROUP BY Sno

      HAVING COUNT(*) >3;

    - HAVING短语与WHERE子句的区别：
      - 作用对象不同
      - WHERE子句作用于基表或视图，从中选择满足条件的元组
      - HAVING短语作用于组，从中选择满足条件的组。
    -

- [例3.48]查询平均成绩大于等于90分的学生学号和平均成绩
- 因为WHERE子句中是不能用聚集函数作为条件表达式，下面的语句是不对的：

```
SELECT Sno, AVG(Grade)
FROM  SC
WHERE AVG(Grade)>=90（WHERE：只有一个元组 不对）
GROUP BY Sno;
```

- 正确的查询语句应该是：

```
SELECT  Sno, AVG(Grade)
FROM  SC
GROUP BY Sno
HAVING AVG(Grade)>=90;
```

  - HAVING写在GROUP BY后面

14 连接查询

嵌套循环法、排序合并法、索引连接

- 等值连接与非等值连接
  - 等值连接：连接运算符为=
  - [例 3.49] 查询每个学生及其选修课程的情况

```
SELECT  Student.*, SC.*
FROM    Student, SC
WHERE  Student.Sno = SC.Sno;
```

  - [例 3.50] 对[例 3.49]用自然连接完成。

```
SELECT Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade（合并了）
FROM    Student,SC
WHERE  Student.Sno = SC.Sno;
```

Student.Sno	Sname	Ssex	Sage	Sdept	SC.Sno	Cno	Grade
201215121	李勇	男	20	CS	201215121	1	92
201215121	李勇	男	20	CS	201215121	2	85
201215121	李勇	男	20	CS	201215121	3	88
201215122	刘晨	女	19	CS	201215122	2	90
201215122	刘晨	女	19	CS	201215122	3	80

- 自身连接：取两个别名，可选择间接先修课等
  - [例 3.52]查询每一门课的间接先修课（即先修课的先修课）

```
SELECT  FIRST.Cno, SECOND.Cpno
FROM  Course  FIRST, Course  SECOND
WHERE FIRST.Cpno = SECOND.Cno;
```

- 多表连接：两个以上的表进行连接

● [例3.54]查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno, Sname, Cname, Grade
FROM   Student, SC, Course   /*多表连接*/
WHERE Student.Sno = SC.Sno
      AND SC.Cno = Course.Cno;
```

15 嵌套查询

基本可以转换为自身连接，多表连接

- 注意子查询不能有ORDER BY，ORDER BY只能对最终查询结果排序
  - 不相关子查询：由里向外逐层处理。
  - 相关子查询：从外到里，重复每一层
- 谓词IN
- 谓词EXISTS
  - 只返回逻辑真假，若为真，就把SELECT的东西送入结果表
  - 不可被其他子查询等价替换，但可替换IN/比较/ANY ALL等子查询

● [例 3.61] 查询没有选修1号课程的学生姓名。

```
SELECT Sname
FROM   Student
WHERE NOT EXISTS
        (SELECT *
         FROM SC
         WHERE Sno = Student.Sno AND Cno='1');
```

- 谓词ANY（SOME）或ALL

- [例 3.59] 查询非计算机科学系中比计算机科学系所有学生年龄都小的学生姓名及年龄。
  - 方法一：用ALL谓词

```
SELECT Sname,Sage
FROM Student
WHERE Sage < ALL
        (SELECT Sage
        FROM Student
        WHERE Sdept= ' CS ')
 AND Sdept <> ' CS ';
```

  - 方法二：用聚集函数

```
SELECT Sname,Sage
FROM Student
WHERE Sage <
    (SELECT MIN(Sage)
    FROM Student
    WHERE Sdept= ' CS ')
 AND Sdept <>' CS ';
```

和多值进行了比较（聚集函数效率更高）

16 集合查询

- UNION并 INTERSECT交 EXCEPT差
  - UNION返回若干个查询结果，但重复时不显示
  - UNION ALL显示重复的

- [例 3.68] 查询计算机科学系的学生与年龄不大于19岁的学生的差集。

```
SELECT *
FROM Student
WHERE Sdept='CS'
EXCEPT
SELECT  *
FROM Student
WHERE Sage <=19;
```

- [例3.68]实际上是查询计算机科学系中年龄大于19岁的学生

```
SELECT *
FROM Student
WHERE Sdept= 'CS' AND
Sage>19;
```